

Master Informatique spécialité Intelligence Artificielle

Réseaux Convolutionnels

15 octobre 2024



Convolution 1D, formule et illustration

Soient deux fonctions mathématiques discrètes f et g . La convolution de f et g , notée $(f * g)$, est donnée par l'équation:

$$(f * g)[n] = \sum_{k=-\infty}^{\infty} f[k]g[n - k] \quad (1)$$

Qui est équivalent à:

$$(f * g)[n] = \sum_{k=-\infty}^{\infty} f[n - k]g[k] \quad (2)$$



Convolution 1D de fonctions bornées

Dans le cas où la fonction g a un ensemble de définition borné sur $[-M; M]$, c'est-à-dire qu'il s'agit d'une séquence de longueur finie, alors:

$$(f * g)[n] = \sum_{k=-M}^M f[n-k]g[k] \quad (3)$$

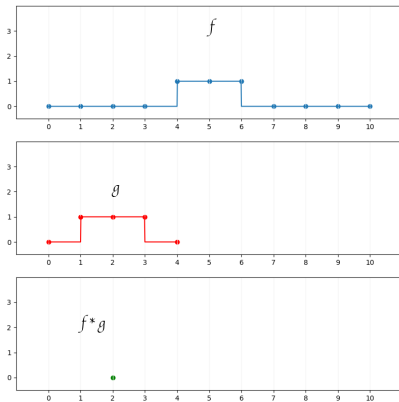


Convolution 1D de fonctions bornées

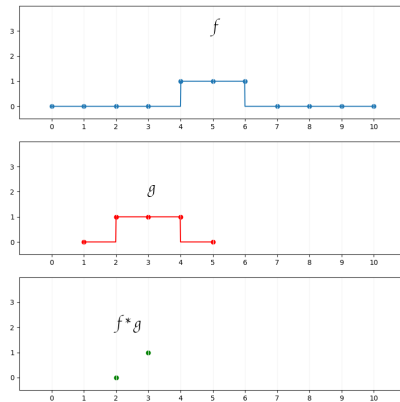
Dans le cas où la fonction g a un ensemble de définition borné sur $[-M; M]$, c'est-à-dire qu'il s'agit d'une séquence de longueur finie, alors:

$$(f * g)[n] = \sum_{k=-M}^M f[n-k]g[k] \quad (4)$$

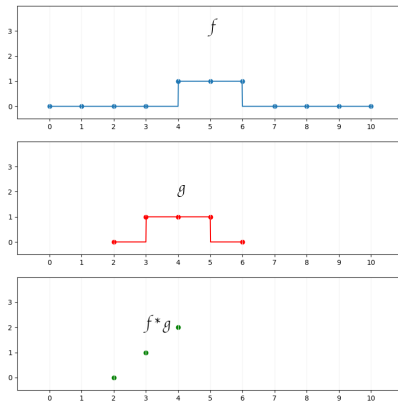
Convolution 1D de fonctions bornées



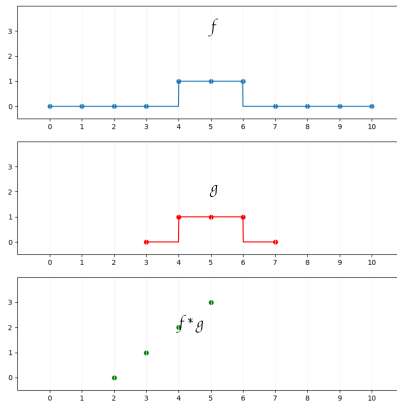
Convolution 1D de fonctions bornées



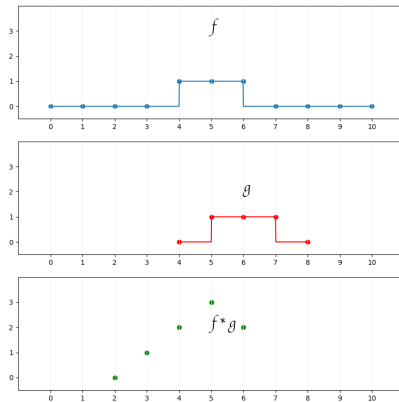
Convolution 1D de fonctions bornées



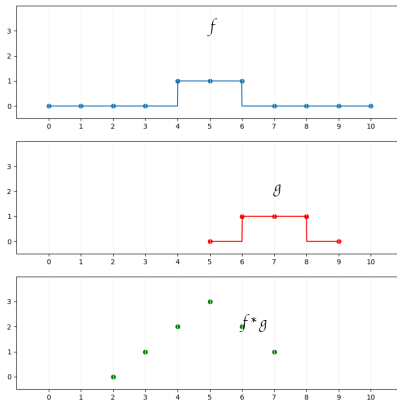
Convolution 1D de fonctions bornées



Convolution 1D de fonctions bornées



Convolution 1D de fonctions bornées





Convolution 2D

Soit deux signaux discrets, f et g , à deux dimensions et définis sur $\mathbb{Z} \times \mathbb{Z}$, alors la convolution $f * g$ est donnée par:

$$h(x, y) = (g * f)(x, y) = \sum_m \sum_n g(x - m, y - n) f(m, n) \quad (5)$$

avec, dans cet exemple:

$$h_{i,j} = \sum_{n=0}^2 \sum_{m=0}^2 g_{n,m} \times f_{i+n,j+m} \quad (6)$$

Convolution 2D



$g_{0,0}$	$g_{0,1}$	$g_{0,2}$
$g_{1,0}$	$g_{1,1}$	$g_{1,2}$
$g_{2,0}$	$g_{2,1}$	$g_{2,2}$

 $*$

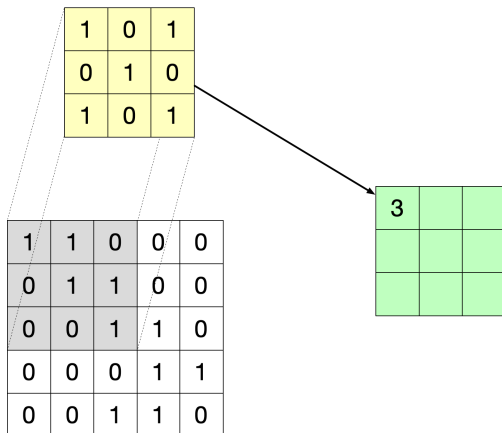
$f_{0,0}$	$f_{0,1}$	$f_{0,2}$	$f_{0,3}$	$f_{0,4}$
$f_{1,0}$	$f_{1,1}$	$f_{1,2}$	$f_{1,3}$	$f_{1,4}$
$f_{2,0}$	$f_{2,1}$	$f_{2,2}$	$f_{2,3}$	$f_{2,4}$
$f_{3,0}$	$f_{3,1}$	$f_{3,2}$	$f_{3,3}$	$f_{3,4}$
$f_{4,0}$	$f_{4,1}$	$f_{4,2}$	$f_{4,3}$	$f_{4,4}$

 $=$

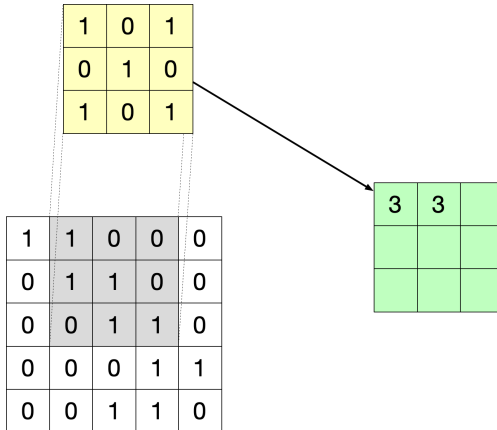
$h_{0,0}$	$h_{0,1}$	$h_{0,2}$
$h_{1,0}$	$h_{1,1}$	$h_{1,2}$
$h_{2,0}$	$h_{2,1}$	$h_{2,2}$

Figure 1: Convolution 2D entre une image de taille 5×5 et un filtre de taille 3×3 .

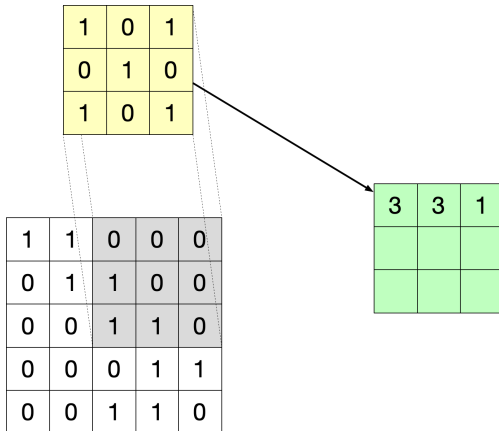
Convolution 2D



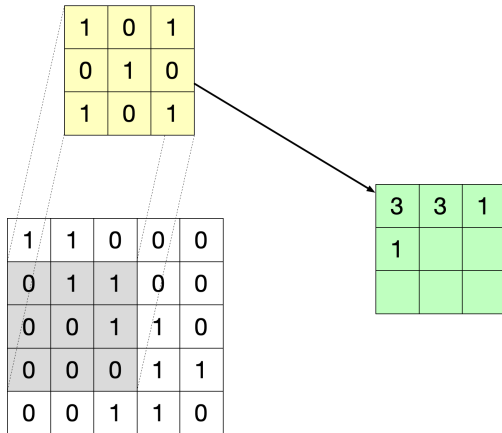
Convolution 2D



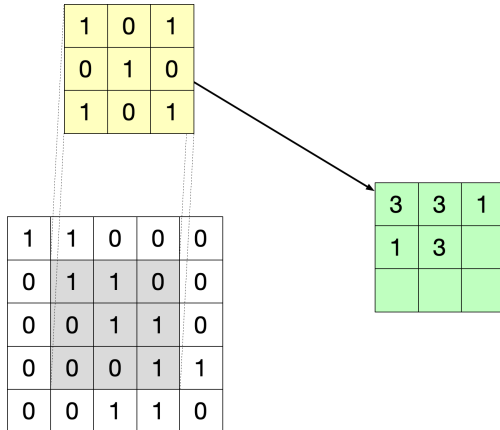
Convolution 2D



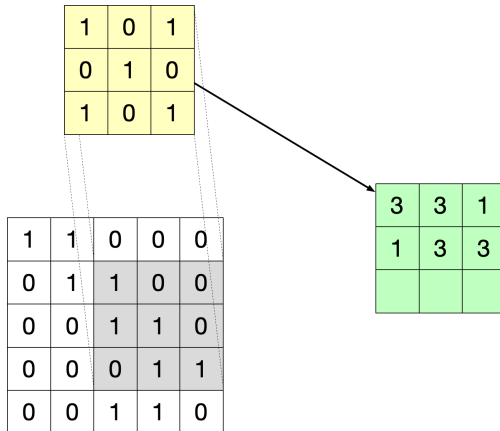
Convolution 2D



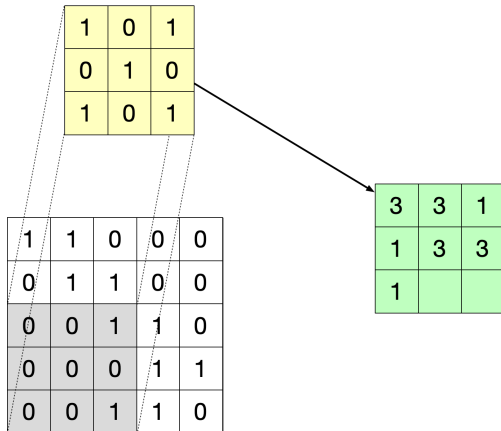
Convolution 2D



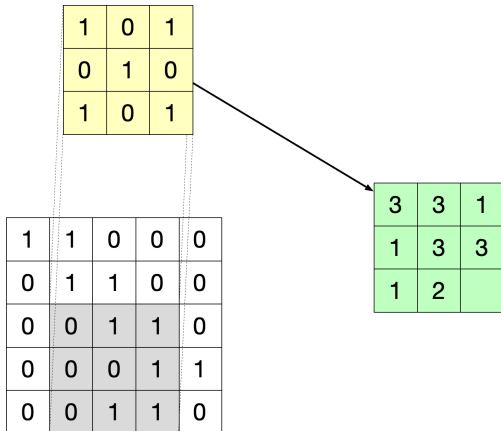
Convolution 2D



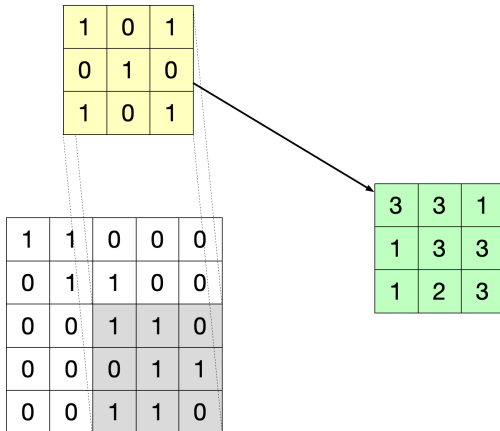
Convolution 2D



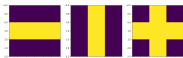
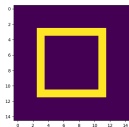
Convolution 2D



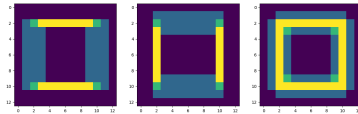
Convolution 2D



Utilisation des convolution 2D



(a) Image à traiter (b) Filtres convolutionels



Les convolutions (qui peuvent être implémentées comme des corrélations) sont depuis longtemps utilisées pour diverses applications comme la détection de contour



Motivations des réseaux de neurones convolutionnels

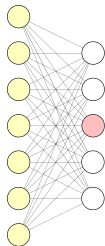
- Les réseaux entièrement connectés ne peuvent pas traiter des données de très grandes dimensions;
- Les CNNs 'utilisent de connections "creuses" entre les couches
- Les CNNs utilisent le partage de paramètres
- Les CNNs permettent une analyse des entrées invariantes

Utiliser des réseaux à matrices de poids creuses

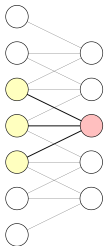


Pour traiter une image en noir et blanc de dimension 28×28 , une couche de neurones à 1000 neurones contient $28 \times 28 \times 1000 + 1000 = 785000$ paramètres.

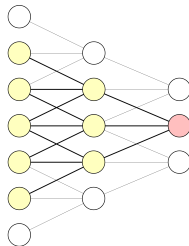
Une image couleurs à 24 millions de pixels $\rightarrow 72.10^9$ paramètres.



Réseau
entièrement connecté
35 poids



Réseau
avec matrice
de poids creuse
15 poids



Réseau à 2 couches
avec matrices
de poids creuses



Utiliser une analyse invariante

Dans un réseau de neurones traditionnel, un paramètre est utilisé exactement une fois lors de la phase *forward*

Dans le cas de réseaux convolutionnels, le poids d'un filtre est utilisé pour traiter toutes les zones "locales" de l'entrée

Le réseau apprend un jeu de paramètre partagé sur l'ensemble des zones de cette entrée

- Les données d'entrées sont "spatialement" corrélées
- Les CNN exploitent la structure des données
- Même traitement est appliqué à toutes les sous-parties de l'entrée → invariance spatiale
- Les CNNs sont inspirés du cortex visuel



Implémentation dans un réseau de neurones



Conv: Implémentation en PyTorch

$$out(N_i, C_{out_j}) = bias(C_{out_j}) + \sum_{k=0}^{C_{in}-1} weight(C_{out_j}, k) \star input(N_i, k) \quad (7)$$

Dimensions de l'entrée: (N, C_{in}, L)

- N la taille de batch
- C_{in} nombre de canaux d'entrée
- L longueur du signal
-

Dimensions de la sortie: (N, C_{out}, L_{out})

- C_{out} nombre de canaux de sortie
- L_{out} longueur de la séquence de sortie



Conv: paramètres en PyTorch

in channels

Nombre de canaux d'entrée

out channels

Nombre de canaux d'entrée

kernel size

- Taille du filtre
- Détermine la taille du *receptive field*, le contexte qui est vu par le réseau, ou en d'autres termes la taille de la fenêtre d'analyse,

Conv: paramètres en PyTorch



Stride

- "Pas" de déplacement du filtre sur la signal lors de la convolution
- Par défaut, *stride* = 1

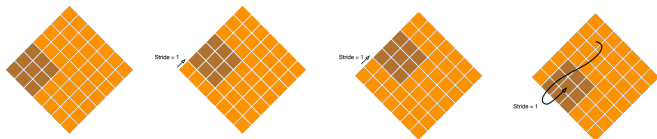


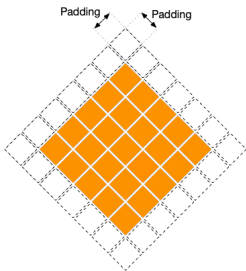
Figure 3: Illustration de l'effet du paramètre *stride* sur une convolution 2D. Sur la première ligne, le pas est de 1, ou plutôt (1, 1) alors que sur la deuxième ligne il est de 2 (ou (2, 2)).

Conv: paramètres en PyTorch



Padding

- Les sorties sont plus petites que les entrées.
- *padding* permet d'ajouter des valeurs au *bord* du signal d'entrée





Conv: paramètres en PyTorch

Les nouvelles variables d'entrée créées avec le padding peuvent prendre plusieurs valeurs:

- 'zeros' par défaut, les valeurs ajoutées sont des zéros.
- 'reflect' ajoute les valeurs d'entrée en miroir
- 'replicate' réplique la valeur du bord du signal dans cette direction
- 'circular' les valeurs sont répétées sur chaque dimension (les valeurs du début sont répétées à la fin et celle de la fin au début)

Conv: paramètres en PyTorch

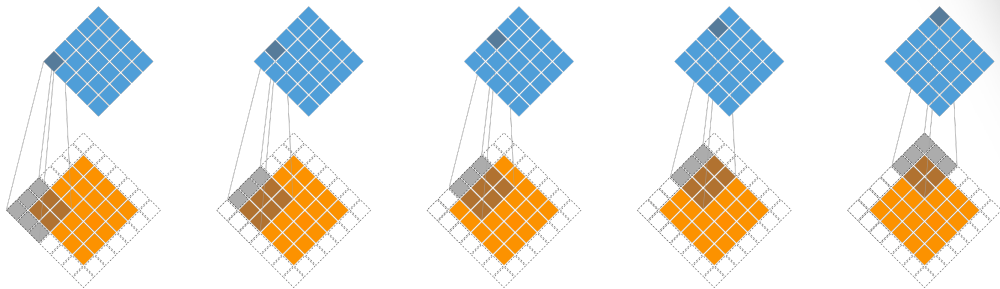


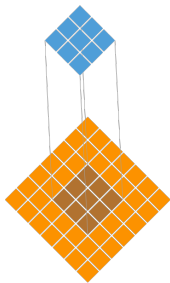
Figure 4: L'utilisation d'une convolution 2D avec un filtre de dimension (3, 3) et un padding de 1 permet de produire une sortie de la même dimension que l'entrée.

Conv: paramètres en PyTorch

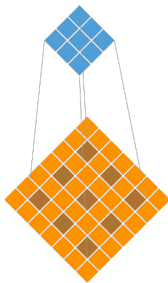


Dilatation

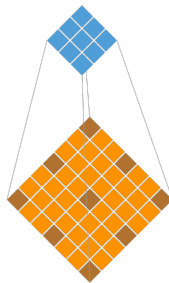
- Données d'entrée sont souvent redondantes
- Valeurs contiguës du signal d'entrée sont corrélées
- On peut réduire la quantité de calcul nécessaire



Dilation = 1



Dilation = 2



Dilation = 3

Conv: paramètres en PyTorch



Grouper

- Permet de différencier les convolutions appliquées sur les canaux d'entrée.
- Peut être utilisé pour implémenter des convolutions *Depthwise separables* (vues plus tard)



Calculer les dimensions de sortie

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times padding[0] - dilation[0] \times (kernel_size[0] - 1) - 1}{stride[0]} + 1 \right\rfloor \quad (8)$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times padding[1] - dilation[1] \times (kernel_size[1] - 1) - 1}{stride[1]} + 1 \right\rfloor \quad (9)$$



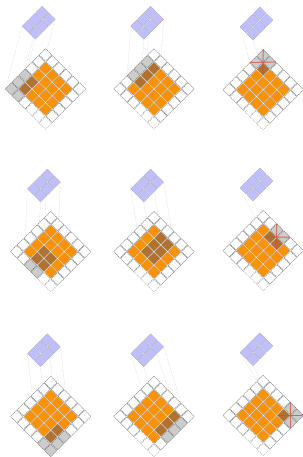
Calculer les dimensions de sortie: exemple 1

Calculer la taille des données de sorties pour:

- taille de l'entrée: $(4, 4)$
- taille du filtre: $(2, 3)$
- padding: $(1, 1)$
- stride: $(2, 2)$
- dilation: $(1, 1)$

Calculer les dimensions de sortie: exemple 1

La dimension de la sortie est $(H_{out}, W_{out}) = (3, 2)$





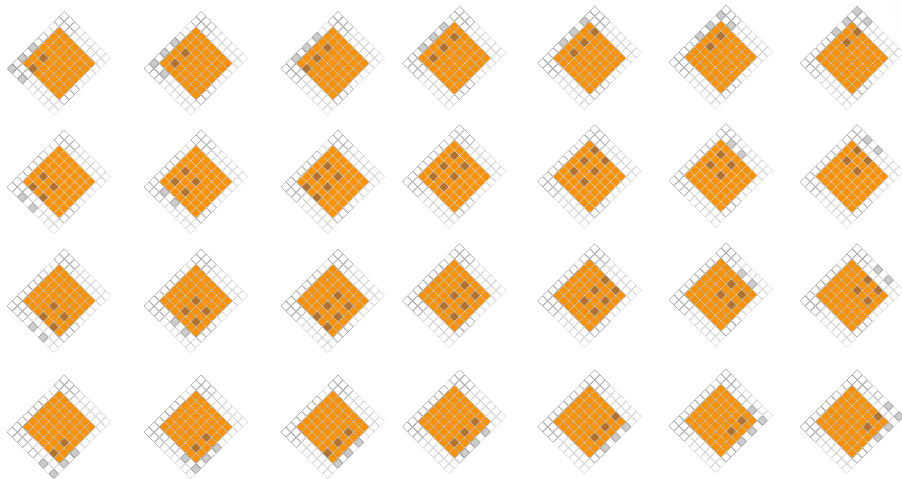
Calculer les dimensions de sortie: exemple 2

Calculer la taille des données de sorties pour:

- taille de l'entrée: $(7, 7)$
- taille du filtre: $(2, 3)$
- padding: $(1, 2)$
- stride: $(1, 2)$
- dilation: $(2, 2)$

Calculer les dimensions de sortie: exemple 2

La dimension de la sortie est $(H_{out}, W_{out}) = (7, 4)$





Nombre de paramètres

Le tenseur de poids du module est de taille:

$$\left(out_channels, \frac{in_channels}{groups}, kernel_size[0], kernel_size[1] \right) \quad (10)$$

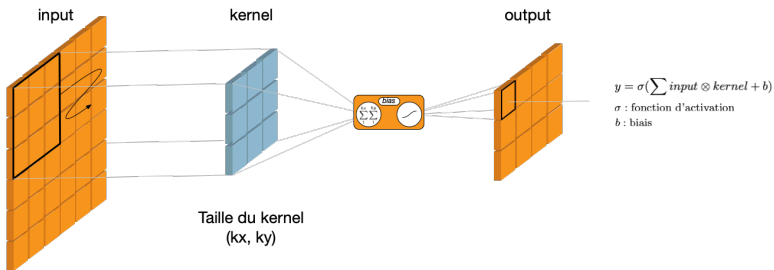
Le tenseur de biais du module est de taille:

$$\left(out_channels, \frac{in_channels}{groups}, kernel_size[0], kernel_size[1] \right) \quad (11)$$

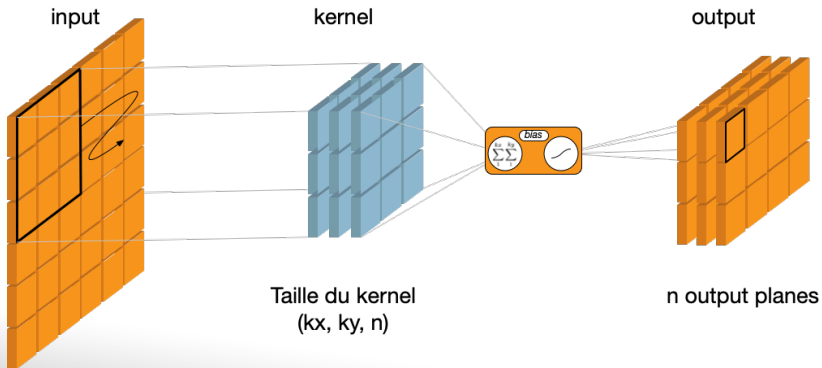
Initialisation des paramètres

Initialisés aléatoirement selon une distribution $\mathcal{U}(-\sqrt{k}, \sqrt{k})$ avec:

$$k = \frac{groups}{C_{in} \times \prod_{i=0}^1 kernel_size[i]} \quad (12)$$



Initialisation des paramètres



$$y = \sigma(\sum input \otimes kernel + b)$$

σ : fonction d'activation

b : biais

Utilisation des biais

- Couches souvent suivies d'une normalisation (ex: BatchNorm)
- BatchNorm inclu un biais
- Effet des biais redondant si les deux couches précèdent l'activation
- Le biais de la couche convolutionnelle est inutile et rajoute du calcul

Souvent on utilise:

Conv -> BatchNorm -> ReLU

```
nn.Conv2d(h_dim, h_dim, 3, stride, 1, groups=h_dim, bias=False),  
nn.BatchNorm2d(h_dim),  
nn.ReLU6(inplace=True),
```

Utilisation des biais

Exemple de rés eau où les biais sont utiles:

```
nn.Linear(64, 32),  
nn.ReLU(),  
nn.BatchNorm2d(),
```





Couches de Pooling



Organisation classique des CNNs

convolution $\rightarrow$ activation $\rightarrow$ pooling.

- Les Convolutions peuvent augmenter fortement la taille des données.
- Pooling $\sim$ sous-échantillonnage qui "résume" le signal.
- Rend la représentation plus invariante par rapport à de petites translations (si on se déplace par une petite translation dans la donnée d'entrée, la sortie ne varie pas ou peu.)

Paramètres des couches de Pooling

- `kernel_size`
- `padding`
- `stride`
- `dilation`





Taille des données de sortie

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times padding[0] - dilation[0] \times (kernel_size[0] - 1) - 1}{stride[0]} + 1 \right\rfloor \quad (13)$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times padding[1] - dilation[1] \times (kernel_size[1] - 1) - 1}{stride[1]} + 1 \right\rfloor \quad (14)$$

Différents types de Pooling: Max pooling

Sélectionne la valeur maximale sur un voisinage.

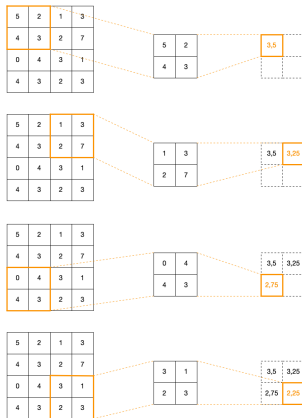


Figure 5: Average-Pooling ou Mean-Pooling à 2 dimensions appliqué à une entrée de taille (4, 4).



Différents types de Pooling: Lp-pooling

Calcule une moyenne après passage à la puissance p de toutes les valeurs sur la fenêtre de pooling

$$f(X) = \sqrt[p]{\sum_{x \in X} x^p} \quad (15)$$

Pooling: une opération multi-dimensionnelle

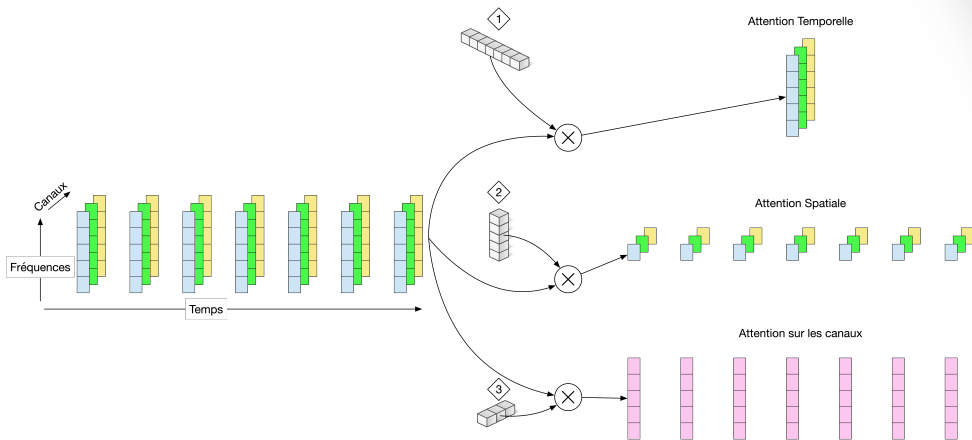


Figure 6: Application d'un pooling appris par un réseau de neurones selon les 3 dimensions d'un signal d'entrée.



Application des réseaux convolutionnels

Classification d'images MNIST



Classification d'images MNIST

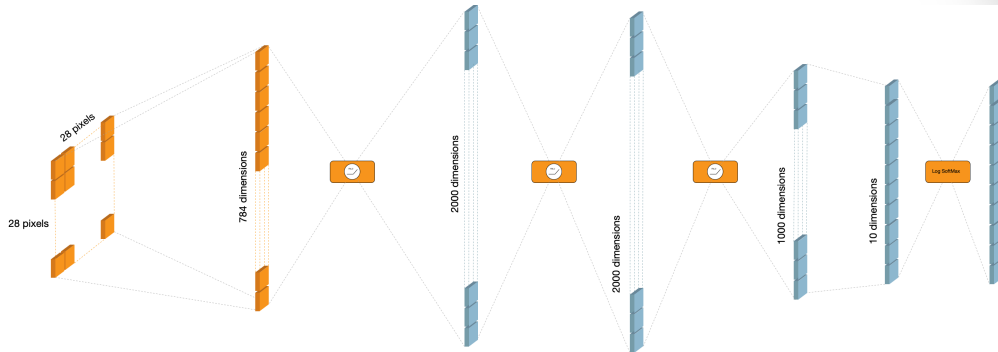


Figure 8: Architecture utilisant des couches entièrement connectées pour la classification de chiffres manuscrits.

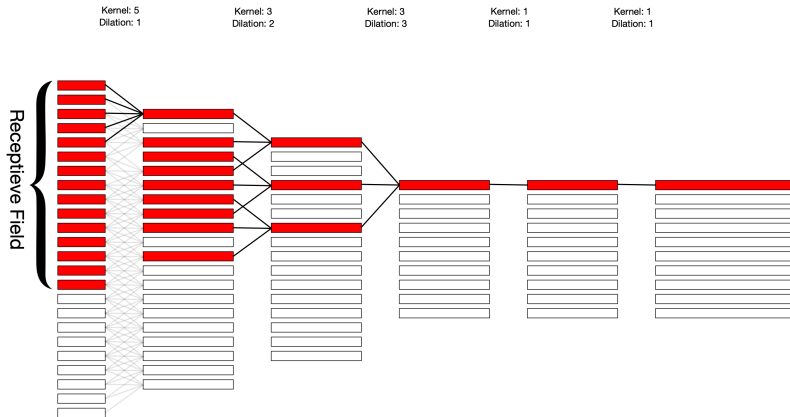
Classification d'images MNIST



```
class CNN(torch.nn.Module):
    def __init__(self, num_classes, activation='ReLU'):
        super(CNN, self).__init__()
        self.conv1 = torch.nn.Conv2d(1, 20, 5, 1)
        self.conv2 = torch.nn.Conv2d(20, 50, 5, 1)
        self.fc1 = torch.nn.Linear(800, 500)
        self.fc2 = torch.nn.Linear(500, num_classes)
        self.relu = torch.nn.ReLU()
        self.pooling = torch.nn.MaxPool2d(2, stride=2)
        self.logsoftmax = torch.nn.LogSoftmax(dim=1)

    def forward(self, x):
        x = self.pooling(self.relu(self.conv1(x)))
        x = self.pooling(self.relu(self.conv2(x)))
        x = x.view(-1, 50 * 4 * 4)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        x = self.logsoftmax(x)
        return x
```

Time-Delay Neural Network (TDNN)



Temporal Convolutional Network (TCN)

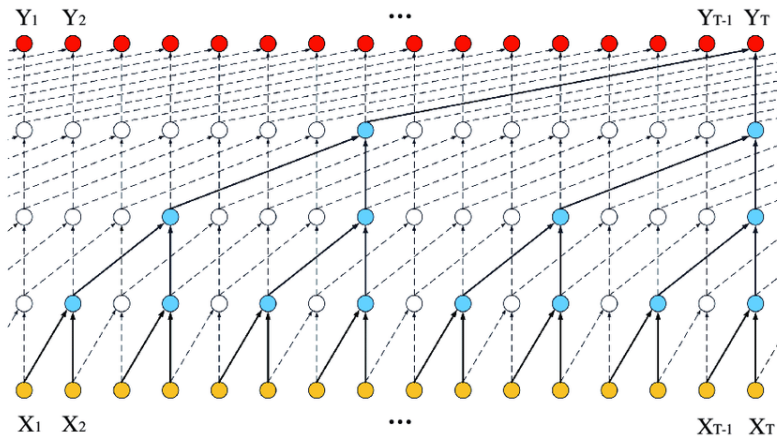


Figure 10: Traitement d'une séquence par un modèle TCN pour le traitement de la parole. (vu le 03/10/24 sur <https://www.researchgate.net>)

ResNet



- Introduit pour le traitement d'images
- Exemple pour la reconnaissance du locuteur

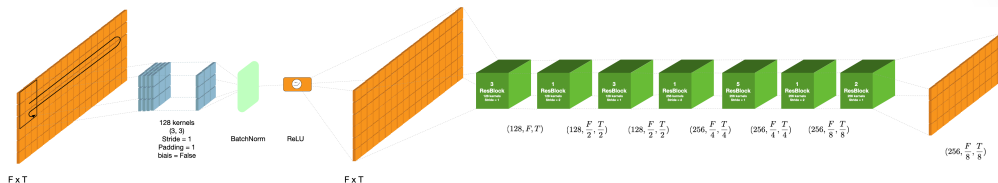


Figure 11: Architecture d'un ResNet 34

Détail d'un bloc de ResNet

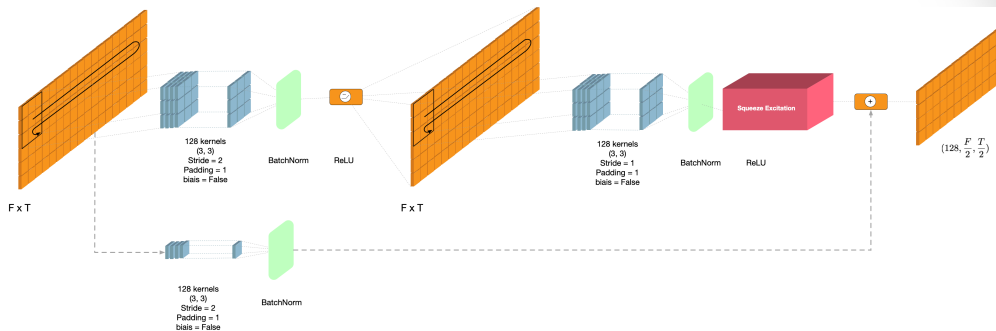


Figure 12: Architecture d'un ResBlock avec 128 filtres de taille $(3, 3)$, avec un padding de 1 et un stride de 2

Détail d'un bloc de ResNet



Détail d'un bloc de ResNet



Squeeze Excitation Layer

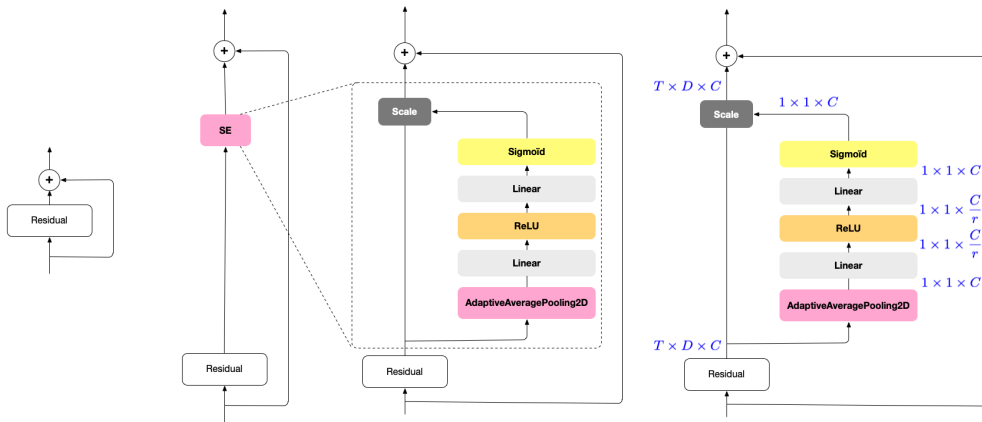


Figure 13: Détail d'une *Squeeze Excitation Layer*

MobileNet



Dimensions d'entrée	Opérateur	t	c	n	s
$224^2 \times 3$	conv2d	-	32	1	2
$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	2
$14^2 \times 64$	bottleneck	6	96	3	1
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1
$7^2 \times 320$	conv2d 1×1	-	1280	1	1
$7^2 \times 1280$	AvgPooling 7×7	-	-	1	-
$1 \times 1 \times 1280$	conv2d 1×1	-	k	-	-

Table 1: Architecture MobileNetV2. t est le facteur d'expansion des blocs Bottleneck, c le nombre de canaux de sortie, n le nombre de fois que le bloc est répété et s le pas de stride de la première couche du bloc. Les couches suivantes ont un stride de 1.

MobileNet: DepthWise separable convolution

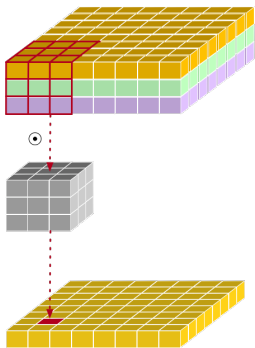


Figure 14: Classic convolution to turn 3 channels into 1 single channel.

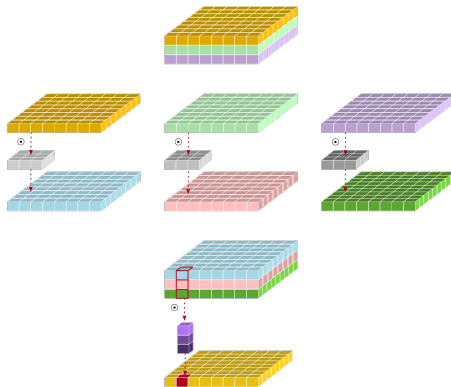


Figure 15: Depth-wise separable convolution to turn 3 channels into 1 single channel.

MobileNet: DepthWise separable convolution

- Moins de paramètres qu'une convolution classique
- Accélère grandement les calculs
- Dégrade très peu les performances
- Limite le sur-apprentissage

```
import torch

class depthwise_separable_conv(nn.Module):
    def __init__(self, nin, nout):
        super(depthwise_separable_conv, self).__init__()
        self.depthwise = torch.nn.Conv2d(nin, nin, kernel_size=3, padding=1, groups=nin)
        self.pointwise = torch.nn.Conv2d(nin, nout, kernel_size=1)

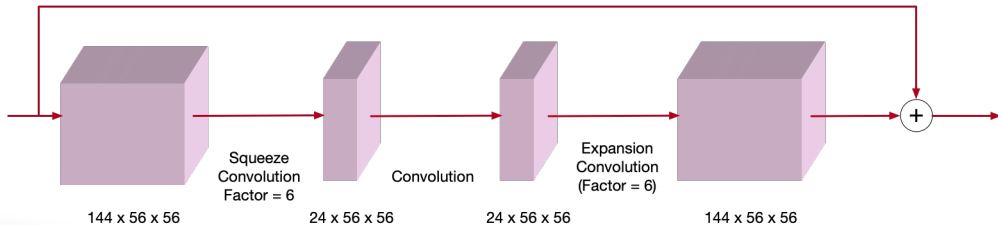
    def forward(self, x):
        out = self.depthwise(x)
        out = self.pointwise(out)
        return out
```

Inverted residual block

Architecture d'un bloc convolutionnel résiduel classique:

wide $\rightarrow$ narrow $\rightarrow$ wide

(16)

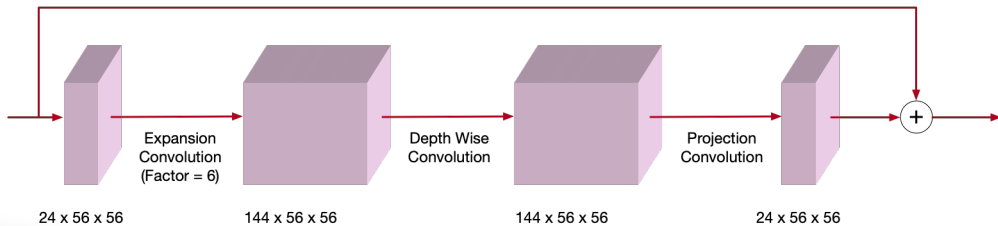


Inverted residual block

Architecture d'un bloc convolutionnel résiduel inversé:

narrow $\rightarrow$ wide $\rightarrow$ narrow

(17)



Block bottleneck

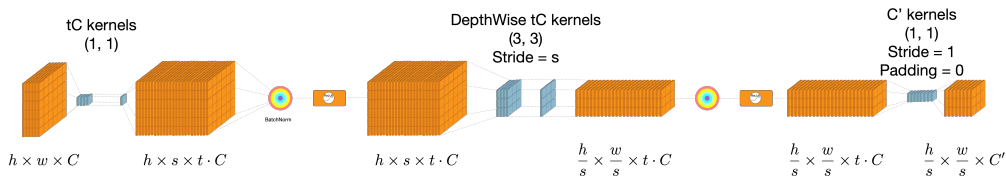


Figure 16: Architecture d'un bloc résiduel convolutionnel bottle-neck utilisé dans les MobileNetV2.

U-Net

Utilisé pour la segmentation d'images.

Architecture globale du U-Net:

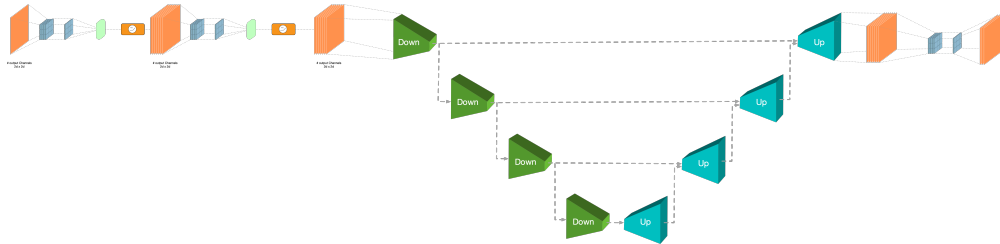


Figure 17: Vue d'ensemble de l'architecture U-Net

U-Net: block down

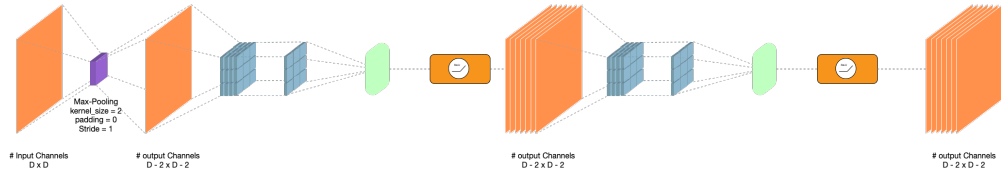


Figure 18: Bloc *Down* de l'architecture U-Net

U-Net: block down

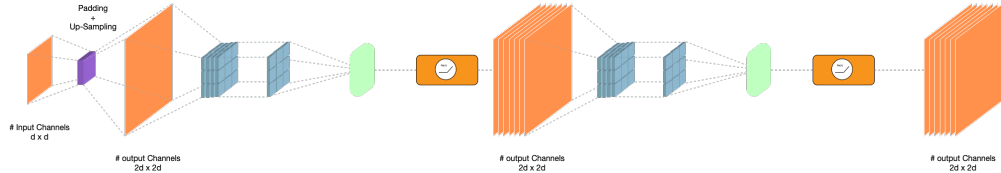


Figure 19: Bloc *Up* de l'architecture U-Net